

J.H. Fletcher & Co. — Pain Points and Demo Ideas

Prep notes for the onsite. Everything here is drawn from the five call transcripts in [Meeting Transcripts/](#). Where I'm inferring rather than quoting, I say so.

1. Source material and chronology

Best-guess order (the files aren't dated; this is reconstructed from internal references):

#	Transcript	Who	Signal that fixes its position
1	Profound_Logic_&_J.H._Fletcher_Check_In	Tiffany, John F, Mike, James, Ray	"a decision was made to move off the IBM i"; TimeClock Plus still in discovery; no ERP selected yet ; Julia described as "the person that's coming in and eventually taking over"
2	Profound_Logic_&_J.H._Fletcher_-_Check-In	Tiffany, Ray, Mike	Timekeeping system now implemented ; Infor CloudSuite Industrial selected ; Akira attack "last year"; CoderFlow just released
3	...Discovery_Call_-_Exploring_Profound_UI_Demo	Jordan, John F, Ray, Mike, James	"Tiffany mentioned CoderFlow to you during your call in May" → #2 was May. ERP decision now un-made — "they're pumping the brakes"
4	Profound_Logic_Demo_-_JH_Fletcher	Gary, John F, Ray, Chuck, Julia , Mike, James	The CoderFlow demo, one week after #3
5	RE...Follow_Up_Items	Jordan + Julia 1:1	Post-demo "inside baseball" call. Board meeting target: second Friday in October

⚠ **Transcript #2 has scrambled speaker attribution.** Large blocks of Tiffany's dialogue are labelled "Mike Odom" and "Ray Henson" and vice versa (e.g. the opening line introducing Juan as "new on the customer success team" is tagged Mike Odom but is obviously Profound-side). Don't quote that file by speaker without sanity-checking it.

The arc that matters: **they had decided to leave the platform, selected Infor, and then Julia pulled the emergency brake.** We are the brake. That's a fundamentally different sales posture from "here's our modernization tool."

2. The room — who actually decides

Julia laid this out explicitly in transcript #5. It's the most valuable thing in the whole set.

Person	Role	Stance	Weight
Rod Duncan	CEO. Julia's first cousin. Purdue CS grad (1980s) — can follow a real technical answer. Likes face-to-face.	Hard over on Infor , "gung ho"	Can veto and kill it. But <i>"he will ultimately be more swayed by Mike and Ray than anything I could say"</i>

Person	Role	Stance	Weight
Julia Fletcher	President. Runs all internal ops. Co-equal owner with Rod. Ex-software: ran ops at a database startup, was a solution designer doing NLP over graph databases.	Our champion. She's the one who said "time out on rip-and-replace"	Equal ownership to Rod, but self-aware that she's counterproductive as the visible pusher: <i>"I'm a little threatening... if he's getting it from Mike and Ray and Matt, then it becomes his idea"</i>
Mike Odom	Business applications developer. 42 yrs on the platform, age 63, retiring soon. Remote (Mississippi), <i>"not physically there a lot."</i>	Most afraid of CoderFlow. Julia's read: late-60s engineer who's always done things one way, worried about retiring without a sustainable system behind him, <i>"hard to hit the I believe button"</i>	Enormous. Rod follows him. He is the single most important person to convert.
Ray Henson	Director of IT. 19.5 yrs. Background is cybersecurity / Windows sysadmin, not RPG.	Engaged, asking good practical questions (hosting, licensing, scheduling, model comparison)	High — Rod follows him too
Chuck Brown	CFO, ~30 yrs. IT reports to him via an org reshuffle. No tech background.	Was "on board with the new ERP." Silent through the demo	Defers entirely to Ray and Mike
James Newman	Developer + IBM i admin. On the platform since '98. 50%+ of his time is sysadmin.	"Really excited about it"	Low formal weight, high credibility with Mike
Matthew Duncan	Rod's son. Junior on the IT team. Purdue CS grad. Julia: <i>"really sharp kid... has the plasticity of mind to do something much bigger here"</i>	Unknown — hasn't been in a call	Strategically enormous. Julia wants him exposed and sold specifically as an indirect channel to Rod
Nick Ruggles	IT, ~30, <i>"hungry and curious"</i>	Unknown	Named by Julia as trainable fast
Ben	Head of sales, titled COO (Julia: a comp-driven title, he doesn't run ops)	Not yet involved	Would need to join later

Julia's own note-to-self, verbatim: *"at the end of the day, to a large degree, you're selling Ray and Mike."*

Company shape: just under 300 employees, **more than two-thirds production/warehouse.** SG&A is very thin. Julia's words: *"I just don't think that we could absorb it, frankly."* That's her central financial argument against the Infor rollout and it should be ours too.

Name discipline: it's **Julia**, not Jennifer — Mike corrected us twice on that call and warned *"it might just squash the whole thing."* Also: Ray is **Director of IT**, Chuck is **CFO**. Julia herself misspoke on this in #5 ("Ray's the CFO") before correcting; don't inherit the error.

3. Where the deal stands, and the sequencing problem

Julia's stated plan, in order:

1. **Put Mike with Lisa** (RPG developer at Schooler running CoderFlow in production), and Ray with Lisa too. Jordan also offered **Brad**, Schooler's app dev manager who led the implementation.
2. *Then* an onsite visit (Jordan offered himself and AJ).
3. Land a **defined one-year engagement / pilot** — that's her expected compromise landing zone.
4. **Present the plan to the board**, second Friday in October (her date was approximate).

The reference calls have to happen before the onsite.

Julia said "step one is really if you can put Mike with Lisa." As of transcript #5 that had not happened. If we walk in before Mike has talked to a peer who does this for a living every day, the onsite is Gary demoing at the skeptic again — which is exactly the pattern that produced the fear spike last time. A developer-to-developer conversation is the thing we cannot substitute for.

Other follow-ups still open from the transcripts:

- **Staff augmentation detail** — Ray asked for it twice ("outside of what's on the website"), and said *"sooner rather than later."* Still owed.
- **Side-by-side model diff view** (Codex vs Claude) — Ray asked during the demo, Gary couldn't find where it moved to. Small, but he asked.
- **Budgetary cost examples** — Ray asked for these back in #3 so the C-suite has a frame *before* sticker shock: *"that way they know we're talking about 1, 2 million versus 20 to 30 million."*
- **Julia's written questions** — she said *"it may be better for me to write things down and pass it through the team."* Worth chasing; they're the real objection list.

The number to beat: Infor wants roughly **\$500K committed over 12 monthly instalments for "Phase 1" — which is really discovery and functional-requirements mapping.** (Standalone discovery is ~\$50–60K.) Julia's reaction: *"do we really want to capex half a million dollars just to do discovery? I don't."* Jordan's counter was that our discovery is 5–6 weeks and then we're *building*. **That contrast is the strongest single argument we have.** It belongs on one page, in writing, in Julia's hands before the board meeting.

4. Pain points

4.1 People and capacity — the binding constraint

- **"One and a half developers."** Mike (full-time apps), James (50%+ consumed by IBM i sysadmin work), two part-time retired consultants who *"I can't depend on to get things done when I need them done"* so they only get non-mission-critical work. Supporting **a couple hundred users** across ERP *and* a homegrown GL/financials.
- **Mike's retirement is the clock on this whole deal.** Julia: *"a lot of the fear at Fletcher right now is driven around Mike Odom's retirement... there's a big fear about him retiring or getting hit by a bus tomorrow."*
- **Reporting bottlenecks entirely through Mike.** *"When someone wants a report, frankly they have to go to Odom to get it."*

- **No knowledge-transfer culture.** James: *"if a person gets replaced, there's not a whole allotment for the outgoing to train the incoming."* When James joined, the outgoing developer had *already retired* and had to come back for 2–3 months. They'd been looking for ~3 years before that. Mike, on James: *"we're still training James."* Mike, on himself: *"I'm still being trained too, honestly."*
- **Hiring is structurally blocked.** Julia: *"I can't get those kind of people to live in West Virginia" + "our company traditionally does not like remote work."* Note the irony that their most critical developer is already remote from Mississippi.
- **Training the user base is its own problem.** Mike: *"training's a big problem with us... it's going to be interesting to see how our people absorb training."* This is his stated reason for doubting the Infor rollout — see §5.
- **Change aversion is cultural, not technical.** Mike: *"this organization is very difficult to change — in fact, they don't even ask for anything, everyone assumes that's how it is... 'if it could have been done, it already would have been done' is their attitude."* People who have never worked anywhere else (one man retiring at 63 who started at 17).
- **No project management vocabulary.** The IT team **doesn't use tickets at all**. Julia: *"they have not ever worked in sprints... our company had never encountered anything other than waterfall planning before I came on board."* She hired a part-time Scrum master for client-facing engineering projects that had languished for *years* with no defined MVP, due date, or budget.
- **Family-business politics.** Julia's own description: *"Game of Thrones meets the Beverly Hillbillies."* Titles don't map to function. Rod holds the veto.

4.2 Technical debt

- **Decades of it, never once addressed.** No RPG conversion project ever ran. Live mix of **RPG II running key systems**, mechanically-converted RPGLE (*"just converted to RPGLE and it's still not very good stuff"*), and some genuinely modern free-format/SQL that Mike has been writing. Mike: *"it's a hodgepodge of things all over the place."*
- **The original authors weren't IT people.** The **retired CEO — an engineer — personally wrote a lot of the key programs**. Others came off the shop floor and went to an IBM RPG class: *"didn't have an IT background, didn't have database training."* Mike is explicit that this isn't a criticism, it's the explanation for the shape of the codebase. It also means **business logic and personal habit are fused** in ways only Mike can currently untangle.
- **Lineage:** 1980s **MAPICS**-derived manufacturing system + homegrown GL/financials + a homegrown time & attendance system **Mike wrote himself**, now back-integrated to cloud TimeClock Plus and ADP payroll.
- **Dirty data / broken integrity.** Program-described files, numeric fields holding invalid data. Mike, describing that same day: a modern-language program *"puked because it had bad data in all these records."* *"Databases are terrible in a lot of places."* Consequence he named: **their existing tools (Abstract, X-Analysis) underperform "because everything isn't connected correctly."**
- **Nobody knows what's live.** *"We've got so much stuff out there that's no longer used and there's not a really good way to track what is and isn't being used. And there's a lot of stuff that IS being used that we've never seen."* They find out by breaking it. **They cannot even estimate lines of code** — Jordan asked directly and got "no."
- **Every change cascades.** *"Anytime we try to make a change, especially a legacy program, it takes us a long time because it's breaking, breaking, breaking, breaking — oh, we didn't know it affected that."* Backlog is roughly **70% improvements / 30% fixes**.
- **A real fear factor about touching things.** Mike: *"you have the capability of cratering the business rapidly if you do something wrong."*

- **Programs that are simply broken or incomplete.** The headline anecdote: one field that *"every so often would get blown away"* — **they worked on it for three years.** It was finally fixed when James pulled it into VS Code and broke it into subprocedures. Mike volunteered this as exactly the class of problem our approach would help with. **Use this program.**
- **Screen sprawl with zero standards.** One menu screen branches to *"90-some-odd different options that everybody uses"* — and **there are eight versions of it:** purchasing, warehouse, inventory, engineering, "this guy wants this version," plus a consolidation attempt from ~10 years ago that *"didn't go over very well"* and yet is still in use. Legacy screens **don't display available command keys** — *"you just have to know."*
- **Manufacturing scheduling is sticky notes.** Literally: *"if you come into our shop and see what our schedule looked like with sticky notes, you'd be shocked at how well a manual thing works."* Their first attempt to feed real system data into scheduling software is what surfaced the database problems.

4.3 The Profound relationship itself — this needs repair work

- **Genie has been shelfware for ~4-5 years.** Bought pre-COVID (2020/21), COVID killed the momentum, *"we've been pretty much sitting on it, not doing much if anything with it at all."*
- **A rewrite project is stranded at 75%.** Started ~2 years ago with Genie in mind. Mike: *"if I sat down and spent all my time on it, I could probably knock it out in about a month."*
- **△ Their most recent Genie upgrade broke their system.** *"We just did an upgrade of the latest version and our system doesn't work anymore — we can't even get Profound to come up right now."* Resolved by PTFs (James downloading them, Ted Holt assisting). **Mike drew a direct conclusion from this about betting on us:** *"that's a scary thing — to think that if we were all committed to that and this happened, we'd be... I don't know where we'd be."* This is unaddressed and it is load-bearing for his skepticism.
- **△ The warehouse scanner problem was never actually solved.** They have **tablets in the warehouse scanning labels** whose values carry prefixes that RPG parses. Tested through Genie ~3 years ago: *"the RPG program didn't get everything that the scanner put in."* It later started working on its own — *"I don't know why, something, the PTF change or something — we didn't really do anything and it started working."* **Never re-tested.** Separately: wide screens present badly on the tablets even in green screen, and nobody has looked at how they'd render in Genie. Mike's own conclusion: they'd need Profound UI for device-aware presentation.
- **Stock Genie flatters the wrong half of their codebase.** Mike's assessment: out-of-the-box Genie works *"really well"* on newer standards-following screens because they follow IBM conventions, and *"they don't fit as well or present as well"* for the legacy ones. Any Genie-first wave must be sequenced with that in mind.
- **In the demo itself, Bob failed authentication on every task.** Gary handled it gracefully and noted Bob is expensive and that Claude performs as well or better — but for an audience already suspicious about *"what's actually there,"* a Profound-branded agent that can't log in is the wrong artifact.

4.4 Business process — the gap Julia cares most about, and the one we answered least well

This was Julia's whole line of questioning at the end of the demo, and we mostly answered it with capability statements rather than a demonstration.

- **Duplicate part numbers.** Her example: *"engineers will create new part numbers because there's so many different ways that the same thing has been named over time and it's too onerous for them to look it up."* Item master pollution, self-reinforcing.

- **People circumvent the system because they can.** Her words: *"some of our process problems are simply that people circumvent the system because they can."* Mike corroborates independently: *"we have problems every single day because they don't follow the system. And there are a lot of controls that we don't have in place that we should have in place."*
- **No cross-department workflow handoffs.** Her example: sales completes a purchase order → should automatically route to engineering for review and **generate a BOM**. Also raised: inventory routing on receipt. Today these are manual/tribal.
- **Data visualization is "a major problem within the company"** — she named it as an explicit pilot goal alongside technical debt cleanup and process streamlining.
- **Friction between engineering / sales / parts / service** as departments. *"I do worry about the way engineering and sales and parts all interact, or service."*
- **Data they're afraid to lose vs. data that may be worthless**, with no way to tell them apart.
- **And she has already rejected the ERP answer to all of this.** This is the key quote in the whole file set:

"There's a lot of magical thinking around 'well, we have these process problems and if we put in Infor, people will have to follow the Infor process and that'll fix it.' And I said no. I worked in software — if you don't work into somebody's workflow, they just ignore it. That's how that works."

She is pre-sold on the thesis. She needs **artifacts to sell it with**, not more persuasion.

4.5 The cyber attack — context that reframes everything

They were hit by **Akira ransomware last year**. A young IT staffer was at work at 6am, saw an alert, thought *"that's weird,"* got a second alert, and **physically pulled cables out of the wall** — with, in his own words to Julia, *"if I'm wrong, I'm going to get fired."*

Recovery was *"massive."* Insured, but it *"cost us a good deal of profit last year."* It delayed the ERP programme by roughly a year.

Julia's takeaway is the strategically important part: it *"really exposed for us the lack of depth we have on the management side — not just the IT side, but frankly just on the management side to manage change within the IT systems."* That is precisely why she doesn't believe they can absorb an Infor implementation right now, and it's a much better argument than any ERP failure statistic.

Implication for the onsite: Ray lived through this. He is a security-first Director of IT with fresh scar tissue. Anything we propose gets evaluated through that lens whether or not he says so.

5. Objections to have real answers for

Julia's two stated questions (transcript #5, verbatim intent)

Q1 — "Are we just kicking the can down the road?"

She has to prove that if the engagement ends — *"either we don't continue or you don't continue, right, either one"* — Fletcher isn't left with a system that's unworkable *"because we've had running over it."*

This is a **reversibility and exit-state** question, not a features question. It needs a demonstrated answer (see Demo A), not a reassurance.

Q2 — "Is this just consulting disguised as a platform? The Palantir model?"

She accepts she'll always need some outside support — *"I'm just not going to staff that kind of talent" —* but wants *"a plan to be able to upskill some of our folks side saddle with yours"* so that ProFound hours land *"at the highest value moments."*

Needs a **named-people, milestone-based handover plan in writing**. Jordan's verbal answer (the Cincinnati customer's "help us for the first 50%") was directionally right but she needs the artifact.

Mike's objection — and why our standard pitch will backfire on him

We have used *"90% of ERP migrations fail"* at least twice across these calls. **Mike has personally been through two ERP migrations and both succeeded.** He said so plainly in transcript #1. He will discount the statistic, and Julia — ex-software — will want a source we haven't cited.

But read what Mike actually said his fear is. It isn't ERP-generic:

"In this particular organization, it's difficult to get people trained on what we have today. They don't follow the system today... I think that's my fear."

His concern is Fletcher's training capacity and process discipline, not ERP as a category. That is *our argument*, and it's stronger than the statistic. Reframe for him: the binding constraint is how much change this workforce can absorb per unit time — and that constraint punishes a big-bang Infor cutover far more than it punishes incremental change to screens people already know. Hand him back his own reasoning. Drop the 90% line in front of him and Julia; use the concrete stories instead (the Cincinnati 18-month failure, the 6-year/\$10–12M one).

Also, and this is the crux of the whole engagement: **Mike is simultaneously the biggest CoderFlow skeptic and the loudest voice for modernization.** He's the one asking to finally turn Genie on (*"minimally we should just give everybody the icon and take away ACS"*), the one who started the stalled rewrite, the one chipping technical debt into free-format and SQL on his own initiative, and the one who says they'll need ProFound UI for the tablets.

So don't ask Mike to change how he personally works. Give him the modernization wins he's been requesting for years and let James, Matthew and Nick be the agentic early adopters. He can endorse the platform without adopting it himself — and his endorsement is what moves Rod. Trying to convert him into an agent-orchestrator at 63, months from retirement, is the one move most likely to lose him.

Ray's questions (he asked these; expect them again, sharper)

- Hosting: on-prem Linux VM vs. ProFound-managed AWS. Who patches it.
- Per-developer Claude accounts vs. server-level keys. Who owns the API key and the spend.
- Model choice and cost control; not being locked to one vendor.
- Scheduling deployments to production for off-hours (his and James's question — display file changes need users off the system).
- **Unasked but certain, given Akira:** what authority does the CoderFlow user hold on the IBM i, what leaves the building, **which LLM provider sees Fletcher source code and under what terms**, what's

the audit trail, what's the network footprint. **Lead with this for Ray, unprompted.** If we make him ask, we've lost the initiative with the person Rod trusts on risk.

Rod's objection (inferred — he hasn't been on a call)

Purdue CS, gung-ho Infor, holds the veto, likes to meet in person, will be moved by Mike and Ray rather than Julia. His likely objections aren't technical capability — they're:

- "We already decided. Why are we relitigating?"
- "Who supports this when Mike leaves?"
- "Does this close the door on Infor?" → **It must not, and we shouldn't want it to.** John already made the right move here in the demo: *"if you were to use CoderFlow, that doesn't mean you couldn't use Infor."* Coexistence is the message that lets Rod say yes without reversing himself in public — which, in a family business with this much title politics, may be the actual blocker.

He has a real CS background and will resent being handled. Show him code and diffs, not slides.

The suspicion problem — read this one carefully

Julia's account of what happened after our demo is the most actionable feedback in the whole set:

"The demo went phenomenally well. Like, amazing demo. People were really excited. And then I'm not entirely sure why, but on the last call there was like this all of a sudden massive fear factor ramp up around it."

And:

"There's just still a little bit of suspicion around, well, that was a really great demo, about what's actually there."

A flawless demo produced fear, not confidence. That's the pattern to break. Concretely, that means:

- **Use their code and their data**, in a read-only sandbox library. In the demo the fleet table was stub data and Ray asked "did it just pull that from our website?" — a small credibility ding that points at exactly this. Generic `cfdemo` customer files will not survive a second viewing.
- **Show something fail on purpose**, and show the recovery: delete the task, container gone, temp library gone, nothing left behind. Mike's real question is "what happens when it breaks." A controlled break answers it; another clean success doesn't.
- **Let them drive.** Hand the keyboard to Matthew or Nick. A junior person succeeding unaided in front of the room is worth more than anything Gary can demo, and it directly serves Julia's plan for Matthew.

6. Demo ideas

Ranked by expected impact on the actual decision. **A** = do before the onsite, **B** = show onsite.

A A. The exit-state demo — "what you own if we both walk away"

For: Julia (Q1), Mike, Rod. The highest-value thing we can build, because it's the one question Julia says she must answer and we've never demonstrated it.

Take one real Fletcher program. Run a change through CoderFlow. Then show the **residue**, deliberately and slowly:

- Plain RPG source, back in a **source physical file** — the way they want it, not just in Git.
- A Git repo **they own**, on their hardware.
- Documentation in a wiki **they own**.
- The object compiled with standard IBM commands, running with no Profound runtime involved.

Then state the honest version: *"cancel us tomorrow and this is all still yours and still works."*

Be honest about the one exception: EJS / Profound UI screens *do* carry a handler dependency. Say so unprompted, then show the green screen still working side by side. Volunteering the limitation is what makes the rest of the claim believable to Mike — and if we don't volunteer it, he'll find it.

A B. The Mike Odom demo — documentation as succession insurance

For: Mike, Julia, Rod. The highest-leverage single move available to us.

Point CoderFlow at the genuinely frightening code — **the three-year-bug program**, or one of the RPG II modules still running a key system. Generate the technical doc, dependency tree, and user guide.

Then invert the demo: **hand Mike the red pen and ask him to find what it got wrong.**

Why this works:

- It reframes CoderFlow from *the thing that replaces Mike* to *the thing that captures Mike* — which is exactly the anxiety Julia diagnosed ("worried about wanting to retire and us not having a sustainable system in place behind him").
- It makes him the **authority** in the room rather than the skeptic. Very hard to stay adversarial toward a tool you've just been asked to grade.
- If the output is good, he says so, and Rod hears it from the person Rod trusts most.

Run this before the onsite so the doc set is already on the table when we walk in — and so we know it's good before we bet the meeting on it.

A C. Close the reference-call loop first

For: Mike, Ray. Not a demo, but it outranks every demo here.

Mike ↔ **Lisa** (RPG developer at Schooler, running CoderFlow in production, *"her whole workflow has changed"*). Ray ↔ Lisa as well. And **Brad**, Schooler's app dev manager, who led the rollout — he's the right match for Ray's and Chuck's "how did you actually govern this" questions.

Julia called this step one. It's still open. See the flag in §3.

B D. Finish the 75%-done rewrite — the cleanest ROI number we will ever get

For: everyone, especially Rod and Chuck.

Mike's own estimate to finish it: **about a month of his undivided time**. If CoderFlow closes a meaningful chunk of that in days, we have an apples-to-apples ROI figure **denominated in Mike's own estimate** — not our marketing. Nothing else in this list produces a number that credible to a CFO.

Ask for the repo as pre-onsite homework. Even a well-defined slice of it is enough.

A E. The dead-code and usage map

For: Mike, Ray, and — critically — the Infor decision either way.

Directly answers *"there's not a really good way to track what is and isn't being used."* Read-only analysis, so it's zero-risk to run and easy to say yes to: object last-used dates, journal analysis, what programs touch what files, cross-reference against the live library list. Deliver a "lean the system" report — what's dead, what runs once a year, what's hot.

The trust play: say out loud that this artifact is valuable to them even if they choose Infor, because scoping an ERP migration requires exactly this inventory and they currently cannot produce it. Being useful to the decision we might lose is the fastest way to stop being a vendor pitching against a competitor.

B F. The guardrails demo — Julia's actual question, finally answered

For: Julia, and it's the ERP-killer if we land it.

She asked how you get ERP-style process discipline without an ERP. We answered in the abstract. Build it instead, on their real item master:

1. **Duplicate part number prevention** — type-ahead on part creation that surfaces "there are already 3 similar items, are you sure?" **without blocking**. Note Gary's instinct here was already right: don't hard-stop the engineer, inform them. A hard block is exactly what gets circumvented.
2. **Validation rules retrofitted** onto an existing entry program — the "controls we don't have in place that we should" that Mike named.
3. **An audit report**: which users are entering invalid data, in which fields, through which programs. Gary already started this live in the demo (customer status values) — productionise it against Fletcher's real data and it stops being a party trick.

B G. The workflow automation demo — PO → engineering → BOM

For: Julia, and for the "Infor gives you a process map" objection.

Build her exact stated example end to end, even with stubs: PO completed in sales → webhook/trigger → task → engineering review queue → BOM generated → notification. Add the **Team Center** hook Ray raised himself (*"we could potentially create that linkage between the 400 and Team Center"*) — that's his own idea handed back to him, which is the most persuasive form of any idea.

This is the concrete rebuttal to *"what an ERP like Infor gives you is a built-in process map."* We can't out-feature Infor's process library. We can show that their **specific** handoff, built to their **actual** process in days, beats a generic best-practice map that people will *"just ignore."*

B H. De-Odom-ify reporting

For: Julia, Chuck, and every business user in the building.

"When someone wants a report, frankly they have to go to Odom." Show a **business user** — Julia herself, ideally, or one of the power users she named (her SVP Sales, her head of HR) — asking for a report in natural language and getting it, then scheduling it to arrive as a PDF every morning.

Two reasons this is stronger than it looks:

- It ties straight to her stated pilot goal that **data visualisation is "a major problem."**
- Julia already observed that *"my users who get the most value are already the most experienced people at the company, because they ask better questions."* So demo it with a senior person. Her own model of how

this technology succeeds predicts that lands — and it puts a non-IT win on the board, which is what makes this a company initiative rather than an IT project.

A I. Retire the warehouse scanner landmine

For: Mike. Non-negotiable.

This is their only concrete, lived, *technical* failure with our product, and it was never solved — it just stopped happening. Mike will raise it. Bring evidence, not reassurance:

- Test the actual label-scan path through current Genie on **their** tablet hardware, verifying the RPG program receives the complete scanner payload including prefixes.
- Show a device-aware/responsive rendering of one of the wide screens that presents badly on tablets today. This also demonstrates the Profound UI value **Mike himself already argued for** — *"we would probably need to step up to Profound UI and know that, hey, this is a tablet."*

Two-thirds of this company works in production and the warehouse. A visible win on the floor is worth more politically than any office-side improvement.

A J. The Genie upgrade postmortem

For: Mike, Ray. Reputation repair.

Their most recent experience of Profound software is *"our system doesn't work anymore."* Before the onsite: verify their install is healthy and current, and bring a short written note — what broke, which PTF levels are required, and a repeatable upgrade-test procedure so it can't recur silently.

Converting our worst recent moment with them into a demonstration of competence buys more credibility than any new feature. Ignoring it leaves Mike's *"that's a scary thing"* standing unanswered.

B K. Turn Genie on — the win Mike asked for

For: Mike, and for immediate visible ROI.

He proposed this himself, twice: *"minimally we should just give everybody the icon and take away ACS."* His reasoning is sound — users who know the keystrokes cold don't need the screens to be pretty, they just need the interface changed once rather than twice, so that later change lands on familiar ground.

Do it with a **Fletcher-branded skin**, and respect his sequencing constraint: first wave = the newer standards-following screens that Genie renders well. Then bring **one** ugly legacy screen fixed as proof the hard cases are tractable.

Framing matters here: this is *"we listened,"* not *"we're changing everything."* It's the cheapest trust-building item on the list.

B L. The eight-screens problem — consolidate presentation, not programs

For: Mike, and every department head.

Eight variants of one 90-option menu, and a consolidation attempt ~10 years ago that *"didn't go over very well"* and is *still in use* alongside the others. That history is the reason to be careful, and also the reason this is such a good demo.

The angle that avoids repeating the 10-year-old failure: don't consolidate the program, consolidate the presentation. One EJS screen, role-driven — purchasing sees the purchasing options, the warehouse sees warehouse options. Nobody's layout is taken away from them, so nobody has to lose. Meanwhile the eight-way maintenance burden collapses.

Also fix the thing Mike flagged: legacy screens *"don't show what buttons are available — you just have to know."* Surfacing available command keys is a small change with a large training payoff, in a company where **training is a named, chronic problem.**

B M. Small but owed: side-by-side model diff

Ray asked, we couldn't produce it. Find where it moved and show it. Delivering on a small ask is a disproportionately strong signal to a Director of IT deciding whether we're reliable.

N. The one-pager Julia takes to the board — not a demo, the deliverable that matters most

For: Julia, Chuck, Rod, and the October board meeting.

Everything above is input to this. One page, side by side:

	Infor Phase 1	Profound Year 1
Cost	~\$500K over 12 monthly instalments	<i>(our number)</i>
What you get for it	Discovery + functional requirements mapping	5–6 weeks discovery, then shipped changes from month 2
First visible result	End of the programme	Weeks
If it goes wrong	Business disrupted; sunk capex	Roll back the phase; green screen still runs
Effect on production	Cutover risk while they're finally hitting on-time delivery	Incremental, opt-in per screen
Does it foreclose the other option?	Yes	No

Include: named upskilling milestones for Matthew, Nick and James (answers Q2), and the exit-state guarantee from Demo A (answers Q1).

Julia is going to argue this in a boardroom against a cousin who can veto it, with a CFO who has no tech background and defers to IT. **She needs one page she can hand across a table, not a deck.**

7. Per-person plan for the onsite

Person	What they need to see	What will lose them
Mike	His own scary program documented correctly (B). The scanner path proven on real hardware (I). The Genie upgrade explained (J). Genie finally switched on, his way (K). Peer conversation with Lisa first (C).	Being told his job is going away. "90% of ERPs fail" — he's seen two succeed. Being asked to become an agent-orchestrator months from retirement. Another flawless demo on fake data.
Ray	Security, authority model, data egress — which provider sees the code and under what terms , audit trail, hosting options, spend control. Off-hours deployment. The side-by-side diff he asked for (M). Staff aug detail, in writing.	Making him ask the security questions himself. Vague answers on where source code goes — he pulled cables out of a wall last year.

Person	What they need to see	What will lose them
Julia	The exit-state demo (A). The guardrails demo (F). PO→engineering→BOM (G). Self-service reporting (H). The one-pager (N) with the upskilling plan named down to Matthew, Nick and James.	More capability theatre. Anything that reads as consulting-with-a-login. Being handed persuasion when she asked for artifacts.
Rod (if present)	Real code, real diffs — he's a CS grad and will resent being handled. Risk and cost comparison. That this does not foreclose Infor. Endorsement voiced by Mike and Ray, not by Julia.	Feeling relitigated or outmanoeuvred. Julia visibly driving. Slideware.
Chuck	Cost, risk, rollback, who's accountable. He has no tech background and will follow Ray and Mike — so the goal is simply that he hears them sound comfortable.	Technical depth aimed at him.
James	He's already sold — use him. Give him something to demo himself. He fixed the three-year bug in VS Code; he's the internal proof that new tooling works here.	Being sidelined as the junior.
Matthew	Hands on the keyboard, launching and completing a task himself. A visible career path <i>"where you're not just a sysadmin."</i>	Being a spectator. Julia's explicit strategic ask is that he's <i>exposed and sold</i> — that requires participation.
Nick	Same as Matthew. Julia flagged him as fast to train.	Not being invited.

8. Traps and do-nots

1. **It's Julia.** Not Jennifer. We got this wrong twice on one call and Mike warned us it could *"squash the whole thing."*
2. **Don't invert Julia's sequence.** References, *then* the visit. See §3.
3. **Don't demo on cfdemo or stub data again.** Ray already noticed the made-up fleet data. Real code, real data, read-only sandbox library.
4. **Don't show Bob,** or verify its credentials 24 hours ahead. It failed every task in the last demo. Gary's own read is that Claude does as well or better at lower cost — so there's no upside and a real downside with an audience that's already suspicious.
5. **Drop the "90% of ERP migrations fail" line** with this group. Mike has personal counter-evidence; Julia will want a citation. Use the specific stories instead.
6. **Never position this as replacing Mike.** Everything is *capturing* Mike. His retirement anxiety is about legacy and sustainability, per Julia — speak to that, not to efficiency.
7. **Don't ask Mike to personally adopt agentic coding.** Let James, Matthew and Nick be the adopters. Mike's endorsement is the deliverable, not his conversion.
8. **Don't let CoderFlow crowd out the commercial asks.** Ray has requested staff-augmentation detail twice and budgetary numbers once. Julia has asked for total cost including consulting. Those are the actual open items; a better demo doesn't close them.
9. **Confirm Mike is physically in the room.** He's remote from Mississippi and *"not physically there a lot."* The single most important attendee joining by video would waste the trip.
10. **Show a controlled failure.** Over-polish is what produced the fear spike. Demonstrate the delete button and the clean teardown.

11. **Watch the scope-control story.** In the demo, Codex *"did more than we asked"* and fixed extra bugs; Gary framed Claude as having done "the right thing." To an anxious audience that reads as *the AI changes things you didn't ask for*. Turn it into the guardrails story it actually is: judges caught it, the human saw it, nothing shipped unreviewed.
 12. **Respect that they want source physical files.** Not everyone wants Git as source of truth, and Gary already flagged the bidirectional-sync work. Don't sell a workflow they've told us they won't adopt.
 13. **Coexistence, always.** Rod needs a path where saying yes to us isn't publicly reversing himself on Infor. Leave that door open on purpose.
-

9. Pre-onsite checklist

- ☐ Lisa ↔ Mike reference call **done** (blocking — §3)
 - ☐ Brad ↔ Ray/Chuck reference call scheduled
 - ☐ Staff augmentation detail sent (owed to Ray, twice)
 - ☐ Budgetary ballpark sent so the C-suite is framed before the conversation
 - ☐ Julia's written question list requested and received
 - ☐ Source dump / repo access obtained for real-data demos
 - ☐ Documentation run completed on the three-year-bug program (B) — and reviewed by us first
 - ☐ Dead-code / usage map generated (E)
 - ☐ Genie install verified healthy; upgrade/PTF postmortem written (J)
 - ☐ Scanner path tested on their actual tablet hardware (I)
 - ☐ The 75%-done rewrite repo obtained; a slice scoped (D)
 - ☐ Security/data-egress brief written for Ray — **provider, terms, authority model, audit trail**
 - ☐ One-pager drafted for the October board meeting (N)
 - ☐ Confirmed: Mike physically present; Matthew and Nick invited; hands-on segment planned
 - ☐ Bob either verified or removed from the run of show
 - ☐ Deliberate-failure segment rehearsed
-

10. Questions to get answered onsite

Things the transcripts don't tell us and that materially change the plan:

1. **Where does the Infor decision actually stand today?** Julia stopped it, but Rod is still "gung ho" and the board meets in October. Is there a signature date we're racing?
2. **What exactly is the 75%-done rewrite?** Which application, what's left, why did it stall beyond time — and can it be the pilot?
3. **Which program is the three-year bug?** Named, so we can document it (B).
4. **Which of the eight screen variants is which,** and who owns each one politically? The 10-year-old consolidation attempt failed for reasons someone still remembers.
5. **What is the actual state of the item master?** How many duplicate/near-duplicate parts? This sizes Julia's top process complaint.

6. **What's on the tablets** — hardware, browser, screen sizes, which programs, how many users?
 7. **How current is the Genie install now**, post-PTF, and did anything stay broken?
 8. **Where does their Git/source control stand?** Gary flagged that getting legacy sources into a repo is the real setup effort. Is there any repo today, or is it all source physical files?
 9. **What did the Akira recovery change** about their change-control, backup and authority model? This shapes everything Ray will accept.
 10. **Who owns the Infor relationship internally**, and what has already been committed on paper?
 11. **What are the two ERP options** they said they were down to, and did Infor CloudSuite Industrial win, or is it still two?
 12. **Is Ben (sales/COO) going to need his own session?** Julia flagged him and the head of commercial as people who'd have to join "down the road."
-

11. The one-sentence version

They have already decided to leave, and Julia stopped it — so our job onsite is not to prove CoderFlow is impressive (the last demo did that, and it produced fear rather than confidence). It's to prove to **Mike** that this makes his retirement safe rather than his career obsolete, to **Ray** that it's secure and doesn't leak their source, and to **Julia** that she has one page and two answers — *we're not stuck with you* and *here's how our own people take it over* — that she can win a boardroom argument with in October.